

✅ 📅 DAY 4 — BGE RERANKER (FINAL VERSION)

🎯 GOAL

Improve retrieval from: BM25 + Vector ❌ (good but noisy)

to:

BM25 + Vector



RRF fusion



BGE Reranker ✅ 🔥



BEST documents only

🧠 WHY BGE RERANKER

👉 Your current system (Day 3):

- retrieves top 10 docs
- but some are:
 - partially relevant ❌
 - noisy ❌

👉 BGE reranker:

(query + doc) → score

- ✅ picks best documents
 - ✅ removes noise
 - ✅ trained for ranking (unlike Qwen)
-

✅ STEP 1 — RERANKER FILE

📁 retrieval/reranker.py

```
# BGE RERANKER (CrossEncoder)
```

```
from sentence_transformers import CrossEncoder
```

```
class Reranker:
```

```
    def init(self):
```

```
        """
```

```
        Load BGE reranker model from local path.
```

```
        IMPORTANT:
```

```
        models/bge_reranker must contain files like:
```

```
        - config.json
```

```
        - pytorch_model.bin
```

```
        - tokenizer files
```

```
        """
```

```
        self.model = CrossEncoder("models/bge_reranker")
```

```
    def rerank(self, query, documents, top_k=5):
```

```
        """
```

```
        query: user question
```

```
        documents: list of retrieved chunks
```

```
        Output:
```

```
        returns top_k most relevant documents
```

```
        """
```

```
        # Step 1: create (query, doc) pairs
```

```
        pairs = []
```

```
        for doc in documents:
```

```
            pairs.append([query, doc])
```

```
        # Step 2: model predicts relevance scores
```

```
        # higher score = more relevant
```

```
        scores = self.model.predict(pairs)
```

```
        # Step 3: combine docs with scores
```

```
doc_score_pairs = list(zip(documents, scores))

# Step 4: sort by score (descending)

ranked = sorted( doc_score_pairs, key=lambda x: x[1], reverse=True )

# Step 5: return only top_k documents

return [doc for doc, _ in ranked[:top_k]]
```

UNDERSTAND THIS FILE

✅ Key idea: (query, doc) → score

Example:

Query: "what is AI"

Doc1 → score 0.89 ✅

Doc2 → score 0.21 ❌

👉 Only keep best

✅ ✅ STEP 2 — UPDATE HYBRID SEARCH

📁 retrieval/vector_search.py

✅ FULL UPDATED FILE

```
from embeddings.embedding_service import EmbeddingService

from embeddings.vector_store import VectorStore

from retrieval.bm25 import BM25Retriever

from retrieval.hybrid import reciprocal_rank_fusion

from retrieval.reranker import Reranker

class HybridSearch:

    def init(self):

        # Embedding model (BGE)

        self.embedder = EmbeddingService()
```

```

# Vector database (Chroma)
self.store = VectorStore()

# BM25 (keyword search)
self.bm25 = BM25Retriever()

# BGE RERANKER (NEW 🔥 )
self.reranker = Reranker()

def index(self, documents):
    """    Store and index documents    """

    # Step 1: generate embeddings
    embeddings = self.embedder.encode(documents)

    # Step 2: create ids
    ids = [str(i) for i in range(len(documents))]

    # Step 3: metadata
    metadata = [{"source": "file"} for _ in documents]

    # Step 4: store in vector DB
    self.store.add(ids, documents, embeddings, metadata)

    # Step 5: fit BM25
    self.bm25.fit(documents)

def search(self, query):
    """    Full retrieval pipeline    """

    # Step 1: VECTOR SEARCH (semantic)
    query_embedding = self.embedder.encode(query)[0]
    vector_results = self.store.query(query_embedding)

    # Step 2: BM25 SEARCH (keyword)
    bm25_results = self.bm25.search(query, top_k=10)

    # Step 3: FUSION (combine both)
    fused_results = reciprocal_rank_fusion(

```

```
bm25_results, vector_results)
```

```
# Step 4: RERANKING (CRITICAL STEP)
```

```
final_results = self.reranker.rerank(  
    query=query, documents=fused_results, top_k=5)  
return final_results
```

COMPLETE FLOW (VERY IMPORTANT)

User Query



Vector Search → meaning match



BM25 → keyword match



Fusion → combine both



 BGE RERANKER → BEST results only



Final Context



Qwen Answer

✅✅ STEP 3 — MAIN FILE

👉 No change required ✅

✅✅ TEST

Run: `python -m app.main`

Test queries:

>> what is artificial intelligence

>> explain machine learning

>> deep learning meaning

✅ EXPECTED IMPROVEMENT

Before	After
mixed quality	clean results ✅
noisy chunks	filtered ✅
average answer	strong answer ✅

⚠️ IMPORTANT CHECK

Make sure this exists: `multimodal_llm/models/bge_reranker/`

Otherwise error: model not found ❌

✅ ✅ FINAL DAY 4 RESULT

You now have:

- ✅ BGE embeddings
- ✅ Hybrid retrieval
- ✅ BGE reranker
- ✅ Fully local system
- ✅ High-quality RAG

🔥 YOUR SYSTEM LEVEL NOW

👉 You are no longer building basic RAG

You built:

- ✅ Production-grade Retrieval System